

UNIVERSITY OF ENGINEERING AND TECHNOLOGY, PESHAWAR

Department of Electrical Engineering

LABORATORY REPORT

EE-170L: Computer Programming Lab

Lab Session 10: More About Functions in C++

Student Name	
Registration No.	BF25NWELE0698
Section	B
Submitted To	Sir Rizwan
Course	Computer Programming Lab

1. Objective

The objective of this laboratory session is to understand and apply three important concepts related to functions in C++: Call by Value, Call by Reference, and Default Values in Parameters. Through hands-on coding exercises, students gain practical experience in how data is passed to functions and how function behavior can be customized using default arguments.

2. Theory / Background

2.1 Call by Value

In call by value, a copy of the actual argument is passed to the function. Any modification made inside the function does not affect the original variable. This method is used when the called function does not need to modify the caller's original data.

2.2 Call by Reference

In call by reference, the address of the actual argument is passed to the function using the '&' (ampersand) operator. The function directly accesses and modifies the original variable in memory. This technique is useful when a function needs to return multiple values or modify the caller's variables.

2.3 Default Values in Parameters

Default parameter values allow a function to be called with fewer arguments than it is declared to accept. If a value is not provided for a parameter during the function call, the default value specified in the function declaration is used automatically. Default values must be specified from the rightmost parameter.

3. Lab Tasks

Task 1: Function with Parameters (multiply)

A function named 'multiply' was written that accepts two integer parameters (num1 and num2), computes their product inside the function body, and displays the result.

Source Code:

```
#include <iostream>
using namespace std;

void multiply(int num1, int num2)    // Function prototype
{
    int product = num1 * num2;
    cout << "Product of " << num1 << " and " << num2
         << " = " << product << endl;
}

int main()
{
    multiply(5, 6);
    multiply(12, 3);
    return 0;
}
```

Expected Output:

```
Product of 5 and 6 = 30
Product of 12 and 3 = 36
```

Task 2: Function with Return Value (getSquare)

A function named 'getSquare' was written that accepts one integer parameter (number), calculates its square, and returns the result to the calling function. The result is then displayed in the main function.

Source Code:

```
#include <iostream>
using namespace std;

int getSquare(int number)    // Function that returns square
{
    return number * number;
}
```

```
}  
  
int main()  
{  
    int num, result;  
    cout << "Enter a number: ";  
    cin >> num;  
    result = getSquare(num);  
    cout << "Square of " << num << " = " << result << endl;  
    return 0;  
}
```

Expected Output:

```
Enter a number: 7  
Square of 7 = 49
```

4. Results and Analysis

Both programs were compiled and executed successfully. The following observations were made during the lab session:

- In Task 1, the multiply() function correctly received two integer arguments, computed the product, and printed the result from within the function using cout. The function had a void return type since output was handled internally.
- In Task 2, the getSquare() function demonstrated returning a computed value to the caller. The result was stored in a variable in main() and then displayed, confirming proper use of the return statement and function return types.
- Both tasks reinforced the concept of call by value, where copies of arguments are passed and the originals remain unmodified.

5. Conclusion

This laboratory session provided practical exposure to function usage in C++. The concept of call by value was clearly demonstrated — function parameters hold independent copies of the passed values, ensuring the original variables remain unaffected. The distinction between void functions (that perform output internally) and value-returning functions (that return computed results to the caller) was successfully explored through the two tasks. Additionally, the role of function prototypes and proper parameter declaration was reinforced. These foundational concepts are essential for building modular, reusable, and maintainable C++ programs.

M. Iqbal Khan

BF25NWELE0698 | Section B

Submitted To: Sir Rizwan

EE-170L | Spring 2026